EDGEFLEET.AI

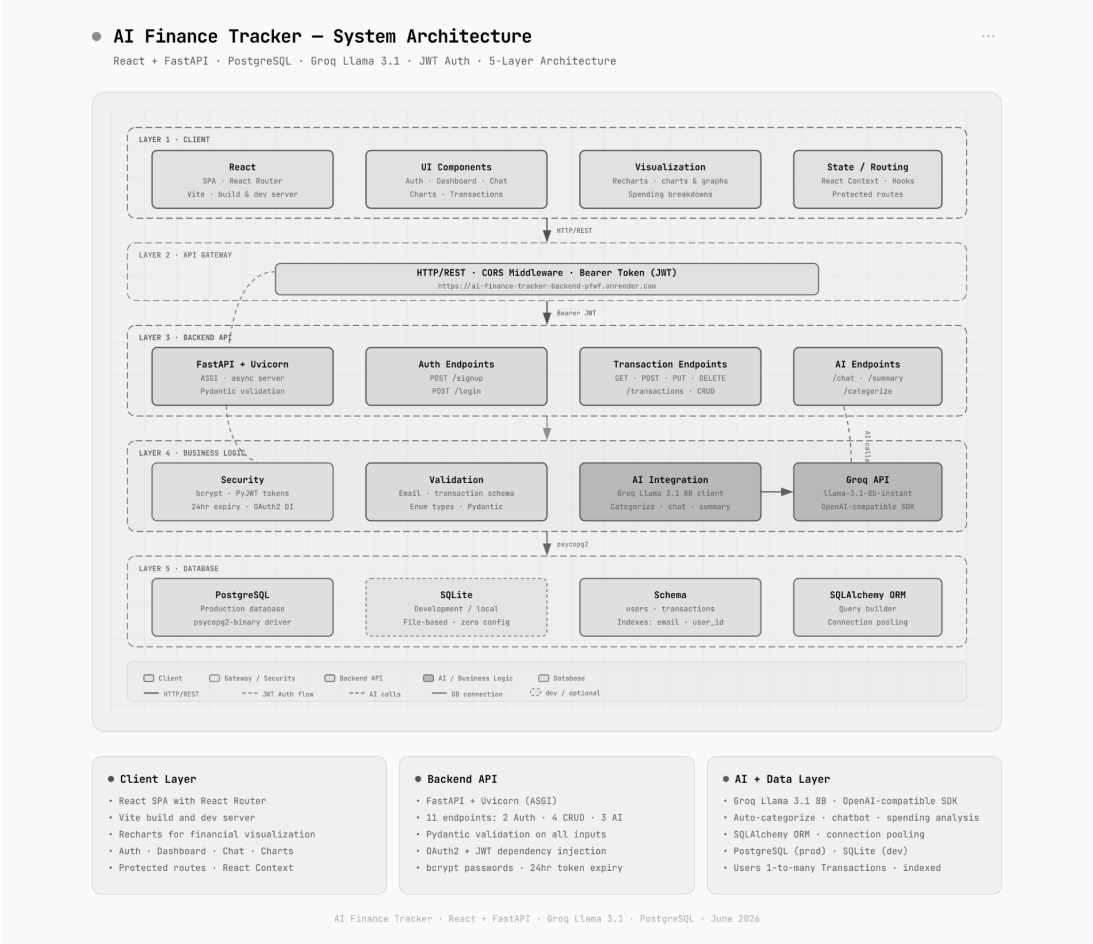
AI FINANCE TRACKER - FULL STACK ASSESSMENT

Submitted by: Bhupathi Sam Gnana Vivek

Discipline: Mechanical Engineering

AI Finance Tracker is a full-stack personal finance management application powered by AI-driven intelligence. Built with Python FASTAPI backend and React frontend, it enables users to create secure accounts, track income and expenses effortlessly. The app integrates Groq's AI API for real-time categorization, conversational chatbot and intelligent financial analysis report.

SYSTEM ARCHITECTURE:



LAYER 1:

I built this visual frontend using **REACT** to create a seamless experience from the secure login page to the main dashboard. This is where users actively engage with the app: viewing live balances, adding or editing transactions, setting monthly budget limits, and interacting directly with AI chat windows.

LAYER 2:

This layer safely transports data over the web using HTTP/REST protocols. I configured CORS middleware and JWT Bearer Tokens here to act as a strict security checkpoint, ensuring only authenticated users can access or change their private financial data.

LAYER 3:

I built this high speed routing layer using FASTAPI to instantly process all incoming actions from the dashboard. I programmed 11 specific endpoints to handle everything the user clicks, from managing secure signups and processing transaction updates(like deleting an old expense) to routing chat questions to AI.

LAYER 4:

In this layer, I implemented strict data validation which uses pydantic modules and password encryption to protect user accounts. Importantly, this is where I integrated the Groq API(Llama 3), allowing the system to automatically categorize new expenses and generate smart, conversational budgeting advice.

LAYER 5:

I set up a cloud-hosted PostgreSQL database (Neon.tech) to safely store user profiles and their entire transaction history. I utilized SQLAlchemy to translate our app's commands into database logic, securely linking every transaction directly to the correct user so their data is ready the moment they log back in.

DATABASE SCHEMA:

Imagine I am using AI finance tracker:

I create an account with email and address, add our transactions, ask an AI chatbot about our transactions and close the app and come back tomorrow. Still, my account exists and transactions exist even after closing the app, AI can analyze the spending history. And importantly, my data is separate from other user's data. This is where databases become really important.

- The database has two main tables:

1. User Table:

This table acts as the secure registry for everyone who signs up. This table includes:

- **id** - unique id for every user
- **name** - the user's full name, allowing the dashboard to personalize UI and greet the user upon login
- **email -** this is the primary login username. The database enforces strict uniqueness so here no two accounts can register with the same address.
- **hashed_password -** passwords are converted into scrambled, irreversible cryptographic string so that even in the event of database breach, user accounts remain completely secure.
- **created_at -** this is a precise timestamp that records the exact moment the account was created, which is critical for system auditing and tracking user retention.

2. Transactions Table:

This table serves as a continuous ledger tracking every single transaction. This table includes:

- **id -** unique tracking number for each specific transaction.
- **user_id** - this is the link that connects this transaction directly back to the id of the user who created it, ensuring strict data privacy.
- **amount** - the exact monetary value of the transaction.
- **type** - it is a label identifying whether the money is moving in or out (income or expense)
- **category -** it is the spending category which our Groq AI engine helps auto-populating, allowing AI categorization.
- **description** - the notes given by the user related to the transaction, using which the category is autopopulated.
- **date -** timestamp capturing exactly when the transaction was logged, allowing Reach dashboard to render precise monthly.

I set up a strict "one-to-many" relationship between user and transaction table. Every single item in the transaction table carries an user's unique ID. This guarantees that when a user logs in, the app instantly filters millions of transactions and only pulls the exact transactions that belong to them.

API STRUCTURE:

The backend operates as the central communication bridge between the React frontend and underlying database/AI engines. Built using FastAPI, the system exposes 11 distinct RESTful endpoints. The API is logically divided into three primary components: Authentication, Transaction management and AI integration.

- **AUTHENTICATION:** To ensure data privacy, the application does not rely on traditional server-side sessions. Instead, it utilized JSON Web Tokens(JWT) for stateless authentication.

POST /signup - This endpoint processes new registrations by first verifying the email is unique, then securely encrypting the password before saving the fresh profile to the database. Then the API sends back confirmation with new user info without hashed password.

POST /login - It verifies the user credentials. Upon a successful match, the API generates a secure access token. This token is sent back by the API as the frontend must attach this token to all the future requests to prove the user identity

GET /users/me - This is a protected route that retrieves the currently authenticated user's profile information to personalize the dashboard, as this API gives us back the name of the user.

- **TRANSACTION MANAGEMENT:** This part handles the continuous flow of financial data. Every endpoint in this category is fully protected, requiring a valid JWT token, The API enforces strict user isolation rules which means it extracts the user's ID from the token and ensures they can only view, edit or delete their own records

POST /transactions - When a user submits a new transaction, the frontend transmits the financial details-including an AI-suggested category according to the description, alongside their secure access token. The API immediately authenticates this token, extracts the user's unique ID to establish strict data ownership and securely logs the new record into the database before returning a successful confirmation ID.

GET /transactions - This retrieves the authenticated user's complete transaction history, allowing the frontend to render lists and charts.

PUT /transactions/{id} - It allows users to safely modify existing records.

DELETE /transactions/{id} - This permanently removes a specified financial record from the database after verifying ownership.

- **AI INTEGRATION:** This set of endpoints differentiates the application from standard ones by dynamically connecting user data to the Groq Llama - 3 AI model.

POST /ai/suggest-category - When a user types a description and type of the transaction, this API queries the AI to instantly suggest the correct budget category. Then this category is returned by API.

POST /ai/chat - This API packages the user's custom question alongside their database records, allowing the AI to provide highly specific answers.

GET /ai/summary - Compiles the user's transaction history and prompts the AI to generate a personalized, conversational summary of their spending habits and potential savings.

When frontend initiates an action, it sends an HTTP request containing the JSON payload and the Authorization token, The API validates this token, process the logic and returns predictable response:

- 200 OK / 201 Created : The action was successful
- 400 Bad request : User submitted invalid data.
- 401 Unauthorized : User's access token is missing or expired, prompting the frontend to redirect them to the login screen.
- 403 Forbidden : A strict security block

AI INTEGRATION EXPLANATION:

The primary goal of this project was to move beyond traditional budget apps. So, there is a use of AI to actively help users understand their money. The application is powered by Groq API, utilizing the Llama 3.1(8b) model. This drives three core features:

1. **Auto - categorization** : This acts like a Data entry assistant. When a user types a description like "Uber" or "Pizza", the system instantly securely sends that phrase to the AI. In milliseconds, the AI contextually understands the purchase and automatically suggests the correct category. This ensures the user's dashboard stays perfectly organized category-wise with almost zero effort.

2. **Instant Financial summary:** This feature acts as a personal financial analyst. At the click of a button, the backend gathers the user's recent transactions and asks the AI to identify patterns instead of just showing totals.

3. **Interactive chatbot:** This is an interactive chat window where users can ask plain language questions. The system securely packages their relevant transactions, feeds it to the AI as context, and returns a direct, conversational answer based entirely on their actual data.

PRODUCT DECISIONS AND TRADEOFFS:

1. **AI : Groq (Llama 3.1) vs OpenAI(GPT - 4):** I chose to power the AI features using Groq API rather than industry-standard OpenAI API.

Tradeoff: There were three major factors here: speed, cost and code flexibility. First, Groq processes requests in milliseconds. Second, Groq's API is currently free, which helped me maintain a strict zero - dollar budget for this project. Third, Groq provides an OpenAI compatible interface, meaning I wrote a standard OpenAI Python code to connect to it. While I traded away GPT-4's slightly deeper reasoning capabilities. I gained extreme speed, eliminated API costs.

2. **Free tier Hosting vs Paid Cloud service:** I deployed the backend on Render's free tier and the frontend on Vercel which are free tier hosting services.

Tradeoff: The major tradeoff of Render's free tier is the **COLD START PENALTY -** the server goes to sleep after 15 minutes of inactivity, cousin the first user login of the data to take 30-60 seconds to load.

3. **Low temperature vs High temperature:** When integrating the Groq Llama-3 model, I deliberately set the AI's temperature to exactly 0.5, avoiding defaults of 0.1 or 0.7+/

Tradeoff: 0.7 is introducing the risk of unpredictable outputs, which is unacceptable for a financial tracker. Conversely, a very low temperature guarantees strict accuracy, making chatbot more robotic and rigid. By tuning the temperature to exactly 0.5, I found the optimal tradeoff, by maintaining a natural, helpful, and conversational tone in the chat interface.

4. **Omitting the "Forgot Password Flow":** For this MVP launch, I consciously chose not to build a "Forgot Password" email recovery system.

Tradeoff: Building a truly secure password reset flow is highly complex. I decided to trade this user convenience feature to keep the project scope focused entirely on perfecting the core AI integrations, data security and cloud deployment within the assignment timeframe.

CHALLENGES FACED:

- The free-tier backend would go to sleep after 15 minutes of inactivity, causing 30 second delays on login.
- The industry standard FastAPI security library is unmaintained and breaks in modern Python versions due to deprecated dependencies. I bypassed this by writing a custom cryptographic implementation using the bcrypt library, manually handling utf-8 byte encoding.
- AI initially returned conversational responses which are not suitable for financial budget planning applications. I resolved this using strict prompt engineering.

FUTURE SCALABILITY CONSIDERATIONS:

- Currently, long-running AI operations (such as generating deep spending summaries) are handled asynchronously within the main web server. To prevent worker starvation under heavy traffic, these tasks should be offloaded to dedicated background workers using a distributed task queue like Celery with a Redis broker.
- As the core ledger grows to millions of rows, standard database queries will slow down. Future iterations will implement horizontal table partitioning in PostgreSQL based on date ranges (e.g., monthly transaction tables) alongside strict composite indexing on `user_id` and `date` fields to maintain sub-millisecond query performance.
- To reduce unnecessary database load, a distributed cache can be introduced to store frequently accessed, slow-changing data—such as user session profiles (`/users/me`) or monthly aggregated charts—directly in memory.
- The current React interface relies on static historical data to render its charts. Scalability will include implementing predictive modeling (using historical spending velocity) to build advanced forecasting visuals, allowing users to interact with interactive predictive lines that simulate how current spending trajectories impact their future monthly budget balance.

BELOW ARE THE SNAPSHOTS OF UI: